

X-shard: Optimistic Cross-Shard Transaction Processing for Sharding-based Blockchains

Jie Xu, Yulong Ming, Zihan Wu, Cong Wang, *Fellow, IEEE*, and Xiaohua Jia, *Fellow, IEEE*

Abstract—Recent advances in cryptocurrencies have sparked significant interest in blockchain technology. However, scalability issues remain a major challenge for wide adoption of blockchains. Sharding is a promising approach to scale blockchains, but existing sharding-based blockchains fail to achieve expected performance gains due to limitations in cross-shard transaction processing. In this paper, we propose X-shard, a blockchain system that optimizes cross-shard transaction processing, achieving high effective throughput and low processing latency. First, we allocate transactions to shards based on historical transaction patterns to minimize cross-shard transactions. Second, we take an optimistic strategy to process cross-shard transactions in parallel as sub-transactions within input shards, thereby accelerating transaction processing. Finally, we employ a cross-shard commit protocol with threshold signatures to reduce communication overhead. We implement and deploy X-shard on Amazon EC2 clusters. Experimental results validate our theoretical analysis and show that as the number of shards increases, X-shard achieves nearly linear scaling in effective throughput and decreases in transaction processing latency.

Index Terms—Distributed system, blockchain scalability, sharding, optimistic concurrency control.

I. INTRODUCTION

RECENT developments in cryptocurrencies have sparked tremendous interest in blockchain technology [1]–[3]. However, scalability remains a major challenge hindering the wide adoption of blockchains. Early blockchain systems, such as Bitcoin [4], can only process up to seven transactions per second, far below the demands of large-scale financial systems [5].

Various solutions have been proposed to increase blockchain throughput [6]–[8]. Among them, sharding has emerged as one of the most promising blockchain scaling approaches. With sharding, transactions and consensus participants (called *validators*) are assigned to different shards. Validators within each shard form a committee to process transactions in parallel with other committees using a classical Byzantine consensus protocol [5]. Each shard maintains its independent blockchain, and the system consists of the blockchains of all shards. Consequently, sharding can increase system throughput linearly with the number of shards in theory [9].

Despite its promise, sharding-based blockchains still face several scalability limitations. First, sharding results in many cross-shard transactions [10]. RapidChain [11] demonstrated that almost all transactions are cross-shard for randomly

assigning transactions to shards based on their identities (IDs) when the number of shards exceeds 16. To maintain system consistency, each cross-shard transaction must be recorded in the blockchains of all involved shards repeatedly. As a result, the *effective throughput*, which is calculated by subtracting the duplicated transactions from the overall throughput, increases much less than the linear gain. The large proportion of duplicated cross-shard transactions recorded across shards severely limits the effective throughput of the whole system.

Second, coordinating across multiple shards introduces high processing latency. Validators do not know account balances in other shards. Thus, a shard must wait for other involved shards to get prepared before processing cross-shard transactions in pessimistic approaches such as lock-then-unlock [9] and two-phase commit (2PC) [12], [13]. While this ensures transaction atomicity, the long waiting time significantly increases transaction processing latency.

Third, it takes a lot of communication and computation resources to prevent malicious attacks during cross-shard coordination. Adversaries may forge messages from other shards to make the system inconsistent. To prevent this, existing schemes [5], [12] require each validator to sign the validation result for each cross-shard transaction and then broadcast it to validators in other shards. The cross-shard communication overhead for each cross-shard transaction is $O(n^2)$, where n is the committee size. A large number of cross-shard messages with signatures occupy network bandwidth, resulting in decreased system throughput and increased processing latency.

To address these limitations, we propose X-shard, a blockchain system that minimizes cross-shard transactions and accelerates their processing. X-shard achieves high effective throughput, low processing latency, and transaction atomicity.

First, we propose a transaction allocation method that allocates transactions to shards based on historical transaction patterns. Accounts that often appear in the same transactions are partitioned to the same shard. Transactions are processed by the shards to which their input and output accounts belong. The number of cross-shard transactions is minimized, and the number of transactions in each shard is kept at a similar level to achieve load balancing across the system.

Second, we propose an optimistic strategy to process Multi-Input Multi-Output (MIMO) cross-shard transactions with low latency. Cross-shard transactions are processed optimistically as if they are all valid and are only checked for validity when they are about to commit in the output shards. We introduce *gateway accounts* managed by committees for processing cross-shard transactions. Each cross-shard transaction is decomposed into multiple intra-shard *sub-transactions* that

J.Xu, Y.Ming, Z.Wu, C.Wang, and X.Jia are with the Department of Computer Science, City University of Hong Kong, Hong Kong. E-mail: jhexu49-c@my.cityu.edu.hk

Manuscript received April 19, 2021; revised August 16, 2021.

transfer coins from user input accounts to the gateway accounts in the input shard. Each input shard optimistically processes sub-transactions in parallel without waiting for other input shards. Any cross-shard transaction that fails validation will be rolled back for transaction atomicity.

Third, we design a cross-shard commit protocol with threshold signatures to reduce communication overhead. Validators in the same shard sign the validation results of cross-shard transactions in batches using threshold signatures. The cross-shard communication overhead for each cross-shard transaction is reduced to $O(n)$.

The contributions of this paper are summarized as follows:

- 1) We propose a transaction allocation method that allocates transactions to shards based on historical transaction patterns to minimize the number of cross-shard transactions and increase effective throughput.
- 2) We propose an optimistic strategy to process MIMO cross-shard transactions. Cross-shard transactions are decomposed into multiple sub-transactions. Each sub-transaction is made within a shard through gateway accounts, which significantly reduces processing latency.
- 3) We design a cross-shard commit protocol that utilizes threshold signatures to batch sign the validation results, reducing the cross-shard communication overhead from $O(n^2)$ to $O(n)$.

The remainder of this paper is organized as follows. In Section II, we present the preliminaries, system model, and problem statement. In Section III, we describe X-shard in detail. In Section IV, we provide a theoretical analysis of sharding-based blockchain performance. In Section V, we analyze the security of X-shard. In Section VI, we present the implementation and performance evaluation of X-shard. In Section VII, we discuss related work. Finally, in Section VIII, we conclude the paper.

II. PRELIMINARIES, SYSTEM MODEL AND PROBLEM STATEMENT

A. Preliminaries

Threshold Signature. A threshold signature is a cryptographic technique that enables a group of nodes to sign a message such that a certain number of nodes must sign for the signature to be valid [14]. A t -of- n threshold signature involves n nodes, of which up to $t - 1$ may be corrupted. At least t of the n nodes in the group must sign the message to create a valid threshold signature [14]. A t -of- n threshold signature satisfies the following properties [15], [16]:

- **Non-forgeability:** It is computationally infeasible for a polynomial-time adversary to generate a valid signature that can be successfully verified by honest nodes.
- **Robustness:** Corrupted nodes cannot prevent honest nodes from generating signatures.

A threshold signature scheme consists of $(TSgen, TSsgn, TScmb, TSver)$ [14], [17]:

- $(sk_1, sk_2, \dots, sk_n; pk) \leftarrow TSgen(1^\lambda)$. Given the security parameter 1^λ , $TSgen$ generates a private key sk_i for each node i and a joint public key pk .

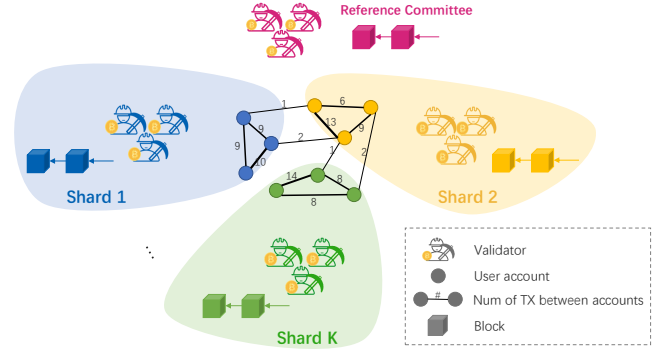


Fig. 1: System model

- $\sigma_i \leftarrow TSsgn(m; sk_i)$. $TSsgn$ takes the message m and private key sk_i of the node i as input, and outputs a partial signature σ_i .
- $\sigma \leftarrow TScmb(\sigma_1, \sigma_2, \dots, \sigma_i)$. $TScmb$ combines the partial signatures $\sigma_1, \sigma_2, \dots, \sigma_i$ to create a joint signature σ .
- $b \leftarrow TSver(m; \sigma; pk)$. $TSver$ takes message m , public key pk and joint signature σ as input and verifies the correctness of the signature σ . It returns $b = 1$ if σ is a valid signature on m . Otherwise, it returns $b = 0$.

Balanced Graph Edge Partitioning. Balanced graph edge partitioning is to partition the vertices of an undirected graph into disjoint subsets such that the sum of the vertex weights of the subsets is approximately equal and the sum of the weights of the cross-subset edges is minimized [18]–[20]. Formally, given an undirected graph $G = (V, E)$, where V is the vertex set and E is the edge set, a balanced graph edge partition algorithm P partitions V into k disjoint subsets $\{V_1, V_2, \dots, V_k\}$:

$$\begin{aligned} & \text{minimize } C(P) = \sum_{e \in \rho(P)} w(e), \\ & \text{subject to } \max_{1 \leq i \leq k} \left(\sum_{v \in V_i} w(v) \right) \leq \frac{(1 + \epsilon)}{k} \sum_{v \in V} w(v), \end{aligned} \quad (1)$$

where $w(e)$ is the weight of edge e , and $w(v)$ is the weight of vertex v , $\rho(P)$ is the cross-subset edge set, and ϵ is an imbalance factor.

B. System Model

X-shard is a blockchain system based on the account model. It consists of K shards and a *reference committee* as shown in Figure 1. The system has a large number of users and N validators, defined as below:

- **Users.** Each user has a user account that belongs to a specific shard. Users generate and sign transactions to transfer coins to others.
- **Validators.** Validators participate in consensus. Each validator belongs to either a shard committee or the reference committee. Each validator has a validated transaction pool and a pending transaction pool for storing validated and pending transactions, respectively.

Each shard has a shard committee responsible for processing transactions. For a committee of shard k , denoted as C_k , it

1) validates transactions whose accounts belong to shard k and adds valid transactions to the blockchain of shard k , and 2) forwards data of processed transactions to the reference committee.

The reference committee is responsible for partitioning user accounts into shards and reconfiguring all committees in the system. More specifically, it 1) collects data of processed transactions, 2) partitions user accounts into K shards according to transaction patterns, and 3) randomly assigns validators to shard committees and a new reference committee.

A cross-shard transaction is a transaction that involves two user accounts that do not belong to the same shard. The shard to which the input account belongs is referred to as the *input shard*, and the committee responsible for processing transactions in this shard is the *input committee*. Similarly, the shard to which the output account belongs is referred to as the *output shard*, and the committee responsible for processing transactions in this shard is the *output committee*.

Each committee consists of n validators, one of which is randomly selected as the leader to coordinate communication in each round. Each validator v has a private key sk_v and a joint public key pk for the t -of- n threshold signature of its committee. The joint threshold signature of the committee is used for committee authentication, with the threshold t set to $\frac{n}{3}$. All validators are assumed to have similar levels of computational, communication, and storage resources.

We consider a slowly-adaptive adversary in line with assumptions made in prior studies [9], [11]. The adversary can corrupt some validators, but the corruption takes some time to take effect. The ratio of corrupted validators against all validators in the system is denoted by β , $\beta < \frac{1}{3}$. Corrupted validators can collude with each other and deviate from the protocol in arbitrary ways, such as sending messages repeatedly, out of sequence, or remaining silent. However, they cannot forge a signature that can be verified by honest validators.

The network model of X-shard is consistent with other sharding-based blockchains [5], [9], [11]. The underlying network is a peer-to-peer network. If an honest validator broadcasts a message, all validators will receive the message within a fixed period of time. Transactions in X-shard do not need to be broadcast to all validators in the network. Users broadcast transactions to any subset of validators in the network, and the validators forward the transactions to the committee that is responsible for processing the transaction using committee-to-committee communication. Committee-to-committee communication is implemented using Kademlia routing mechanism [21] as in RapidChain [11]. This allows a validator to communicate with a logarithmic number of committees to discover the committee responsible for processing the transaction [11].

C. Problem Statement

We aim to design a blockchain system that effectively addresses the challenges of cross-shard transaction processing in sharding-based blockchains. The system shall have the following characteristics. 1) The system should minimize the number of cross-shard transactions while ensuring a balanced

transaction allocation across multiple shards to maximize effective throughput. 2) The system should process cross-shard transactions optimistically in parallel to reduce transaction latency. 3) The system must guarantee that either all parts of a cross-shard transaction are successfully executed, or none are, thereby eliminating inconsistencies and ensuring transaction atomicity.

III. DESIGN OF X-SHARD

A. Overview of X-shard

X-shard proceeds in fixed intervals called *epochs* (e.g., a day). Each epoch consists of a reconfiguration period and a consensus period. The process is illustrated in Figure 2.

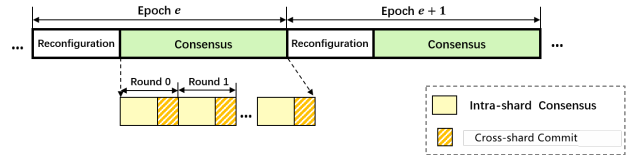


Fig. 2: An illustration of the process of X-shard

During each reconfiguration period, the current reference committee randomly assigns validators to a reference committee and K shard committees for the next epoch. Validators within each shard negotiate keys for threshold signatures, and the gateway accounts for other shards. The reference committee adds gateway accounts and the validators' IDs for the next reference committee to the reference blockchain.

User accounts are partitioned into shards at specified intervals (e.g., once a year) to optimize system performance. If the specified period has elapsed since the last account partitioning, the current reference committee re-partitions user accounts into K shards according to their historical transaction patterns. The updated account partitioning results are also added to the reference blockchain.

During each consensus period, shard committees validate transactions and add valid transactions to blockchains. A consensus period includes multiple rounds, with each round consisting of two main phases: intra-shard consensus and cross-shard commit.

- Intra-shard consensus phase: Each shard committee validates transactions belonging to their shard. Valid transactions are then added to the shard's blockchain.
- Cross-shard commit phase: Shard committees exchange gateway account balances and the latest processed sub-transactions and report this data to the reference committee. If an output committee receives all validated sub-transactions for a cross-shard transaction from all input committees, it adds the cross-shard transaction to its blockchain. If validation fails, the output committee generates rollback transactions to roll back received commit sub-transactions of the failed cross-shard transaction to ensure atomicity.

In the following subsections, we first present the user accounts partitioning. Then, we introduce operations for cross-shard transaction processing: optimistic cross-shard transaction processing, threshold signature for cross-shard commit, and the rollback mechanism.

B. User Accounts Partitioning

Many sharding-based blockchains randomly assign transactions to shards based on transaction IDs [9], [11], [22]. For example, transactions with IDs ending in 0 are allocated to shard 0, and those ending in 1 are allocated to shard 1. While this approach can keep the number of transactions assigned to shards at a similar level, it inadvertently causes a large number of cross-shard transactions [11]. Cross-shard transactions need to be recorded repeatedly across multiple shards, reducing the effective throughput.

To address this issue, X-shard uses a transaction allocation approach based on historical transaction patterns. The goal is to minimize cross-shard transactions and improve effective throughput. X-shard focuses on the transaction-level allocation and can be used with other validator-level assignment solutions to further improve blockchain scalability and security.

In X-shard, accounts that frequently interact are partitioned into the same shard to minimize cross-shard transactions. Each shard is responsible for processing transactions involving accounts partitioned to that shard. To enable such account-based allocation, X-shard constructs an account relationship graph, as illustrated in Figure 3. In this graph, user accounts are represented as vertices. An edge exists between two accounts if they have transacted before, with the edge weight indicating the number of transactions. The vertex weight reflects the total number of transactions related to that account.

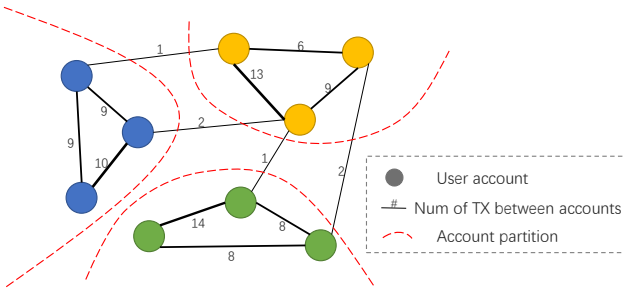


Fig. 3: Account relationship graph partitioning

During the consensus period of epoch e , the reference committee continuously collects transaction data from K shards. The transaction data includes the accounts and the number of transactions between accounts in the latest block. Based on the collected transaction data, the reference committee incrementally updates the account relationship graph built in epoch $e - 1$.

Additionally, the account relationship graph is periodically re-partitioned into the K shards every pre-defined time period T since the last partitioning. If time period T has elapsed since the last account partitioning, the leader of the reference committee uses Metis [18], a classic balanced graph edge partition algorithm as mentioned in Section II-A, to re-partition the updated account relationship graph.

Specifically, Metis partitions the graph into K non-overlapping groups of accounts, denoted as $V_1, V_2, \dots, V_K$. Each group of accounts is assigned to one of the K shards. Metis minimizes the number of edges between the groups,

which helps minimize cross-shard transactions. It also balances the number of transactions in each group, making the transaction load equal across shards. Metis is a reliable algorithm that can partition even very large social networks with hundreds of millions of nodes [23]. Its partitioning results have been extensively analyzed. The details of Metis and other partitioning algorithms are out of the scope of this paper.

The leader broadcasts the updated account relationship graph and the account partitioning results to other validators in the reference committee for verification. Once validated, the new account relationship graph is added to the reference blockchain using Practical Byzantine Fault Tolerance (PBFT) [24]. In subsequent epochs, validators refer to the latest partitioning results to forward and process transactions.

User accounts may migrate across shards when re-partitioning. To ensure consistency, migrating accounts are frozen to halt new transactions. The source shard committee settles outstanding balances, rolls back unfinished transactions, and signs the final account state. This signed status is transferred to the target shard, where the account resumes operations after being unfrozen. Further details on specific migration protocols and mechanisms to prevent attacks will be provided in future work.

The re-partitioning interval, while suggested yearly based on observed stability, can be customized based on factors such as hardware capabilities and network conditions to keep cross-shard transactions below a threshold. The separation of user account and validator partitioning allows independent configuration based on their distinct optimization purposes.

During system initialization, when historical transaction data is not yet available, user accounts are first randomly allocated to different shards. Once a sufficient amount of transactions have been processed after the initial phase, accounts are then re-partitioned into shards based on transaction patterns collected by the reference committee.

C. Optimistic Cross-shard Transaction Processing

In order to prevent one shard from processing a cross-shard transaction while another shard is processing a conflicting transaction, existing schemes [12], [13] take pessimistic methods. They require a shard to wait for other involved shards to get prepared before processing cross-shard transactions. The pessimistic methods result in increased transaction processing latency and decreased concurrency, as other transactions are prevented from accessing the account balances involved until the transaction has been processed.

X-shard adopts an optimistic strategy. Each input shard first treats received cross-shard transactions as valid in other input shards, and validates them. This allows cross-shard transactions to proceed concurrently in input shards. Then, each output shard checks for conflicts when the cross-shard transaction is about to commit. Because transactions are not required to wait for all input shards to be prepared, X-shard can achieve higher concurrency and better performance than pessimistic methods.

X-shard introduces gateway accounts to decompose cross-shard transaction validation into multiple intra-shard validation. Gateway accounts are dedicated accounts created and

managed by shard committees specifically for processing cross-shard transactions. Since gateway accounts are collective accounts held by committees and do not belong to any users, we do not set incentive mechanisms for them. Each committee creates $K - 1$ gateway accounts, one for each other shard, as shown in Figure 4. The committee that creates a gateway account also holds the private keys of the gateway account, which is generated as described in the threshold signature scheme [15]. The operations of gateway accounts need to be authorized by the threshold signatures of their holder committees when processing and validating cross-shard transactions. Specifically, the committee C_j in shard j maintains gateway accounts $\{G_j^k | 1 \leq k \leq K, k \neq j\}$ for other shards. G_j^k denotes the gateway account corresponding to committee C_k in shard j processed by committee C_j . If a transaction has G_j^k as an input account, it is valid only if the threshold signature of committee C_k in the transaction is valid. All gateway accounts are added to the reference blockchain during the reconfiguration period.

For a cross-shard transaction involving m input shards, users generate m sub-transactions, one for each input shard. The input accounts of a sub-transaction are user input accounts of the cross-shard transaction in an input shard, and the output accounts are the gateway accounts of the output committees in the input shard. These sub-transactions are processed in parallel in their respective input shards. Input committees validate the sub-transactions and forward the valid ones to output committees.

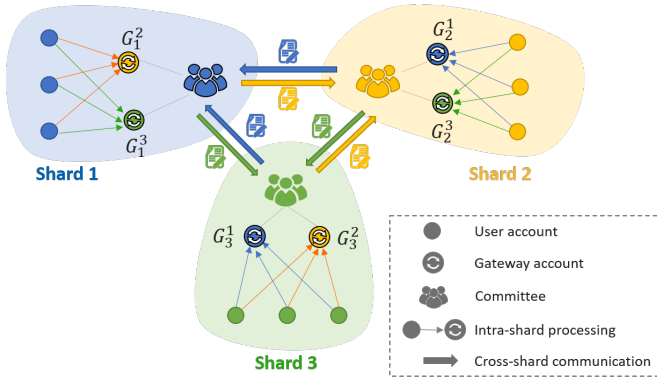


Fig. 4: Cross-shard transaction processing

For example, consider a cross-shard transaction tx with input accounts $a_1, a_2, \dots, a_j$ in shards $1, 2, \dots, j$ and output accounts $a_k, a_l, \dots, a_m$ in shards $k, l, \dots, m$ as follows:

$$tx : a_1 + a_2 + \dots + a_j \rightarrow a_k + a_l + \dots + a_m. \quad (2)$$

Its sub-transactions are:

$$\begin{aligned} sub_1 : a_1 &\rightarrow G_1^k + G_1^l + \dots + G_1^m, \\ sub_2 : a_2 &\rightarrow G_2^k + G_2^l + \dots + G_2^m, \\ &\dots \\ sub_j : a_j &\rightarrow G_j^k + G_j^l + \dots + G_j^m, \end{aligned} \quad (3)$$

where $G_j^k, G_j^l, \dots, G_j^m$ are the gateway accounts corresponding to shard $k, l, \dots, m$ respectively in shard j .

Users broadcast both the cross-shard transaction tx and the sub-transactions $sub_1, sub_2, \dots, sub_j$ simultaneously. The

cross-shard transaction tx is forwarded to all input and output shards, while each sub-transaction $sub_1, sub_2, \dots, sub_j$ is forwarded to the corresponding input shard $1, 2, \dots, j$, respectively.

If validators of input shard j receive sub-transaction sub_j within time Δ after receiving the cross-shard transaction tx , they are optimistic that other shards also receive tx and corresponding sub-transactions, and validators of input shard j validate sub_j . If sub_j is valid, it is added to validated transaction pools and added to the blockchain later.

The parameter Δ is the validity period for cross-shard transaction processing. This prevents output shards from waiting indefinitely if they only receive some but not all valid sub-transactions from input shards. If an honest user broadcasts a cross-shard transaction and sub-transaction at time t , then input shards will receive the cross-shard transaction and corresponding sub-transactions within $t + \Delta$. If a validator of input shard j fails to receive the valid sub-transaction sub_j within time Δ after receiving the cross-shard transaction tx , the validator adds the cross-shard transaction's ID to its abort lists $B_j^k, \dots, B_j^m$. The abort list B_j^k contains IDs of cross-shard transactions that need to be rolled back whose input shard is j and output shard is k .

The committee leader packs the transactions in its validated transaction pools along with its abort lists $B_j^1, \dots, B_j^K$ to generate a new block. Validators in the same shard validate the block and add the validated block to their blockchain via the intra-shard consensus protocol PBFT.

D. Threshold Signatures for Cross-shard Commit

Shards coordinate cross-shard transaction processing in the cross-shard commit phase after the intra-shard consensus phase. Output committees only commit a cross-shard transaction once they receive validation results from all input committees. Previous schemes [5], [12] rely on all validators of the input committee to sign each validation result independently and send validation results to output committees. Their cross-shard communication overhead for each cross-shard transaction is $O(n^2)$, where n is the committee size. In X-shard, input committees use threshold signatures to sign validated sub-transactions in batches. They then send the sub-transactions and joint threshold signatures to output committees, reducing cross-shard communication overhead to $O(n)$.

Specifically, after each new block is appended to the blockchain, a validator v_i in input shard j signs $\{h_j, g_j^k, L_j^k, B_j^k\}$ with its private key sk_i for each output shard (e.g., shard k), as shown in Eq. (4).

$$\sigma_i \leftarrow TSsgn(h_j \| g_j^k \| L_j^k \| B_j^k; sk_i), \quad (4)$$

where h_j is the block height of the latest block of shard j . g_j^k is the current balance of the gateway account G_j^k corresponding to shard k . L_j^k is a list of sub-transactions in the latest block involving G_j^k and their cross-shard transaction IDs. For example, (sub_j, id) is included in L_j^k if sub_j is in the latest block. B_j^k is the latest abort list of cross-shard transactions in the latest block for shard k .

Validators broadcast their partial signature σ_i within their committee. If a validator of input shard j receives no less than $\frac{n}{3}$ partial signatures for $\{h_j, g_j^k, L_j^k, B_j^k\}$, it combines the signatures as in Eq. (5).

$$\sigma \leftarrow TScmb(\sigma_1, \sigma_2, \dots, \sigma_i) \quad (5)$$

The validator then generates the cross-shard message $M_j^k = \{h_j, g_j^k, L_j^k, B_j^k, \sigma\}$ and broadcasts it to output shard k . Similarly, it generates and broadcasts cross-shard messages to other output shards.

If a validator of an output committee C_k receives all valid sub-transactions of a cross-shard transaction tx , the validator adds tx to its validated transaction pool so that tx will be added to the blockchain in shard k . Similarly, tx is added to blockchains of all output shards $k, l, \dots, m$, and the cross-shard transaction processing ends.

E. Rollback Mechanism Achieving Atomicity

Sub-transactions are validated independently in input shards, and validation results are sent to the output shards by input shards, respectively. However, for a cross-shard transaction with multiple inputs, some of the inputs may be valid while others are not. This can lead to a lack of atomicity, as some input shards may validate a cross-shard transaction successfully while others do not. Some works, such as RapidChain [11], are not atomic because they ignore rollbacks for cross-shard transactions that are only valid on some shards. To address this issue, X-shard uses a rollback mechanism to ensure the atomicity of cross-shard transactions. If an output shard cannot receive sub-transactions from all input shards, it generates rollback transactions to roll back the sub-transactions it has received.

Specifically, when a validator in output shard k receives an abort list that includes a cross-shard transaction's ID id , it means the validation of the cross-shard transaction tx in an input shard has failed. The validator in output shard k cannot receive all sub-transactions of tx . In this case, it generates a rollback transaction R_j for each received sub-transaction sub_j of the cross-shard transaction tx as shown in Eq. (6).

$$R_j : G_j^k \rightarrow a_j. \quad (6)$$

The validator signs R_j with its private key sk_i as shown in Eq. (7) and broadcasts $\{R_j, \gamma_i\}$ to other validators in the output shard k .

$$\gamma_i \leftarrow TSsgn(R_j; sk_i). \quad (7)$$

If a validator in output shard k receives no less than $\frac{n}{3}$ partial signatures for the transaction R_j , it combines them to get the joint signature γ as in Eq. (8).

$$\gamma \leftarrow TScmb(\gamma_1, \gamma_2, \dots, \gamma_i) \quad (8)$$

Finally, the validator broadcasts the rollback transaction R_j with the joint signature γ to input shard j .

If validators of input shard j receive the rollback transaction R_j , they validate the joint threshold signature γ . If valid, the rollback transaction is added to their validated transaction pools and will be added to the blockchain in input shard

j . Thus, the sub-transaction optimistically processed in input shard j will be rolled back. If all rollback transactions for the cross-shard transaction from output shards are added to the input shard blockchains, the processing of the MIMO cross-shard transaction ends with an abort.

IV. PERFORMANCE ANALYSIS OF SHARDING-BASED BLOCKCHAINS

In this section, we develop a theoretical framework based on queuing theory to analyze the performance of sharding-based blockchains. While previous research has primarily focused on evaluating overall transaction throughput [8], we examine the overlooked yet critical metrics of effective throughput and transaction processing latency.

A. The Model for Theoretical Analysis

We consider a blockchain system with k shards. The system is modeled as an open Jackson queuing network [25], and each shard is modeled as an M/M/1 queue [25]. Each committee is regarded as a server, transactions are regarded as customers, and the service refers to adding transactions to the blockchain. Each queue is served by one server, and customers are served on a first-come, first-served basis.

We define *effective transactions* to evaluate the actual transaction demands of users, and *effective transaction throughput* and *transaction processing latency* to evaluate the effective performance of the sharding-based blockchain system.

- *Effective transactions* refer to intra-shard transactions and cross-shard transactions, excluding sub-transactions, rollback transactions, and repeated copies of cross-shard transactions on blockchains.
- *Effective transaction throughput* refers to the rate at which the system processes effective transactions. The processing of an effective transaction is completed if and only if all involved shards have completed processing the transaction.
- *Transaction processing latency* refers to the time it takes for a transaction to be added to the blockchain from when it is received by the shard it belongs to. It includes the interval during which transactions are held in transaction pools and the interval required for validators to process these transactions.

We make the following assumptions for our model based on classical queuing theory [25]: 1) The time for a committee to process a transaction is independent of the transaction type and follows an exponential distribution with the parameter μ . 2) The rate at which effective transactions arrive at the system can be modeled using a Poisson process with parameter λ . 3) The time it takes for transactions to be forwarded to the relevant shards is negligible. Transactions will wait in a queue if the committee is currently busy. 4) Accounts are randomly assigned to shards. Each committee only processes transactions whose accounts are assigned to its shard.

To facilitate theoretical analysis, we make simplified assumptions of random account allocation in this section. This differs from the actual allocation in X-shard, but allows deriving cross-shard transaction probabilities. The simplification

enables formal proof of sharding scalability and explains the principles without specific allocation effects. It offers a baseline understanding of sharding's impact, independent of any specific account transaction mechanisms. Moreover, this model sets a foundational benchmark for evaluating the performance of X-shard. By isolating the effect of sharding from other factors, our simplified assumptions enable a focused study on how sharding improves system scalability.

B. Effective Transaction Throughput of Sharding System

Theorem 1. *The maximum effective transaction throughput of the system grows linearly with the number of shards.*

The theorem demonstrates that the maximum effective throughput of the system, excluding duplicated cross-shard transactions, grows linearly with the increase in the number of shards. This implies that the impact of redundant records of cross-shard transactions on the throughput of a sharding-based blockchain system is negligible when the number of shards is sufficiently large. For a comprehensive proof of this theorem, refer to Appendix A.

C. Transaction Processing Latency of Sharding System

Theorem 2. *The average transaction processing latency of the system decreases as the number of shards increases.*

This theorem proves that as the number of shards increases, the transaction arrival rate at each shard decreases. Increasing the number of shards can lead to faster processing of transactions on average. The full proof is presented in Appendix B.

The above theoretical performance analysis proves the effectiveness of sharding in improving the performance of blockchain systems even without an optimized transaction allocation mechanism. The optimized transaction allocation mechanism, as described in Section III-B, can further improve the efficiency of the system by ensuring a balanced transaction allocation and minimizing the number of cross-shard transactions.

V. SECURITY ANALYSIS

A. Committee Security

X-shard focuses on transaction allocation, while committee security depends on validator assignment. In this section, we prove the security of X-shard when adopting a random validator assignment scheme, such as those in Omniledger [9] and RapidChain [11]. Specifically, we will demonstrate that the proportion of malicious validators in any committee does not exceed $\frac{1}{3}$.

Theorem 3. *For each epoch with an unbiased and random committee formation algorithm, as long as $\beta < \frac{1}{3}$ and the number of validators in the committee n is sufficiently large, any committee is honest with an overwhelming probability.*

The proof of Theorem 3 is given in Appendix C. Therefore, the committee security of X-shard is guaranteed if the number of validators in each committee is sufficiently large.

B. Security of Cross-shard Commit

Any cross-shard message $M_j^k = \{h_j, g_j^k, L_j^k, B_j^k, \sigma\}$ from committee C_j to C_k contains the threshold signature of C_j . Because the number of malicious validators in a committee is less than $\frac{n}{3}$ and the threshold of the threshold signature is $\frac{n}{3}$, any malicious validator cannot forge cross-shard messages with valid signatures. If the threshold signature of the cross-shard message is valid, at least one honest validator must participate in the signature. This means that information contained in cross-shard messages is trustworthy, so g_j^k is the current balance of the gateway account G_j^k , L_j^k is the list of sub-transactions in the latest block in shard j , and B_j^k is the latest abort list for shard k .

A rollback transaction R_j^k for cross-shard transaction tx from C_k to C_j is valid only if its threshold signature is signed by no less than $\frac{n}{3}$ validators in C_k . Because the number of malicious validators in C_k is less than $\frac{n}{3}$, at least one honest validator is involved in generating the rollback transaction R_j^k . This means that at least one honest validator has received an abort list containing the cross-shard transaction tx from an input shard. The processing of the cross-shard transaction tx will end with an abort, and all received sub-transactions will be rolled back.

C. Atomicity of Transactions

Each committee is responsible for processing transactions that involve accounts assigned to its shard. Intra-shard transactions, which involve input and output accounts belonging to a single shard, are only processed by the committee of that shard. If the transaction is valid, it will be added to the blockchain of the shard; otherwise, it will be discarded. Similarly, sub-transactions and rollback transactions, which involve input and output accounts belonging to a single shard, are only processed within a shard.

Suppose a cross-shard transaction tx involves multiple input shards and multiple output shards. If an output committee C_k added the cross-shard transaction tx to its blockchain, it means that C_k has received all valid sub-transactions from input shards, and all input committees have processed sub-transactions of tx . According to Theorem 3, all committees are honest. The input committees also send sub-transactions to other output committees. Other output committees will also process tx and add it to their blockchains. Therefore, once an output shard processes a cross-shard transaction, it means that the cross-shard transaction has been optimistically processed by all input shards and will be processed by all output shards.

On the other hand, if a cross-shard transaction tx fails to be processed in an output shard, it means that the output shard received an abort list containing id . Because committees are honest, input shards must broadcast id to all involved shards. Other output committees will also receive an abort list containing id . As a result, the cross-shard transaction tx is not added to any blockchain by any output committee, and all output committees generate rollback transactions to roll back sub-transactions that have been optimistically processed in input shards.

VI. EXPERIMENTAL EVALUATION

To evaluate the performance of our proposed X-shard system, we implemented a prototype in Golang and conducted experiments on Amazon EC2 t2.micro instances. Each instance is equipped with 1 vCPU, 1G RAM, and 1G memory. The nodes in each shard are connected to a P2P network, and each node has three peer nodes on average. We add a propagation delay of 100 ms on each link to simulate the communication of geographically distributed nodes. The actual bandwidth of nodes is around 880 Mbps. The code of X-shard is available at <https://github.com/my17/xshard>.

A. Number of Cross-shard Transactions

In this section, we present the experimental evaluation of our transaction allocation method, which aims at minimizing cross-shard transactions. Our method's effectiveness and efficiency are demonstrated through a comprehensive experiment involving real-world Ethereum transactions [26], encompassing a dataset of 248 million transactions and 35.9 million accounts as provided by Xblock [27].

We initiate our evaluation by constructing an account relationship graph utilizing the transaction data. This graph is subsequently divided into shards using Metis [18]. Transaction allocation to specific shards is achieved by each shard processing transactions associated with accounts that have been allocated to it.

We compare our method with a baseline that randomly assigns accounts to shards based on transaction IDs [9], [11], [22]. For a sample of 1,000,000 randomly selected Ethereum transactions, we analyze and compare the number of cross-shard transactions under both our account partitioning strategy and the baseline strategy. The results are shown in Figure 5.

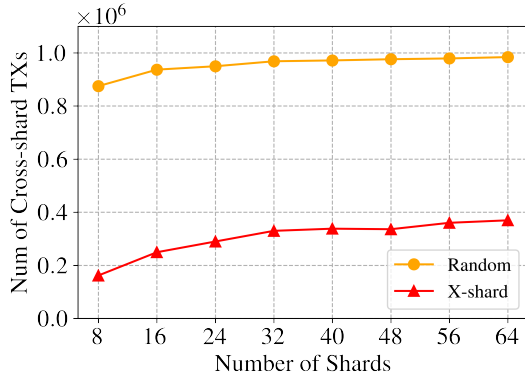


Fig. 5: Comparison of numbers of cross-shard transactions

We can observe a trend where the number of cross-shard transactions increases with the number of shards in both methods. When the number of shards in the system is greater than 16, the baseline method results in almost all transactions being cross-shard transactions. This result is consistent with the theoretical analysis in RapidChain [11]. In contrast, as the number of shards increases, the number of cross-shard transactions in our proposed X-shard scheme only increases slightly.

Furthermore, our method consistently outperforms the baseline, maintaining significantly fewer cross-shard transactions, irrespective of the number of shards. For example, when the number of shards is 8, the number of cross-shard transactions in X-shard is only one-fifth of the number of cross-shard transactions in the schemes with random transaction allocation. This clear discrepancy underscores the effectiveness of our proposed transaction allocation method in substantially reducing cross-shard transactions.

B. System Throughput

System throughput and transaction processing latency are two important metrics to evaluate the scalability of a blockchain system. In this subsection, we evaluate the impact of various system parameters on the throughput of X-shard.

We implement a threshold signature scheme for gateway accounts based on the practical threshold signatures proposed by Shoup *et al.* [15]. The gateway account generation time and signing/verification cost have minimal impact on system performance (less than 2% of cross-shard transaction latency), but gateway accounts enable major reductions in coordination overhead by transforming cross-shard transactions into independent intra-shard operations, as analyzed in Section III-D.

We deploy a TX-driven simulator in the network to simulate users generating Ethereum transactions and broadcasting transactions to the network. By default, the TX simulator broadcasts 1 million real Ethereum transactions provided by Xblock [27] each time, and our blockchain system processes 1 million transactions each time. The average transaction size is 114 bytes. The maximum block size is constrained by the maximum number of transactions that a block can contain.

First, we evaluate the impact of the number of shards on system throughput. The transaction arrival rate is fixed at 3000 transactions per second (tps). The committee size is set to 10 validators and the maximum block size is 100 transactions. Figure 6a shows the overall transaction throughput, effective transaction throughput, and cross-shard transaction throughput of the system at different numbers of shards. As expected, all three types of system throughput increase as the number of shards increases. When the number of shards is between 8 and 40, the effective transaction throughput increases approximately linearly with the number of shards. It is consistent with the theoretical analysis in Section IV. When the number of shards is greater than 40, we observe the throughput increases slightly as the number of shards increases. After further analysis, we found that a large number of blocks failed to reach the maximum block size when the number of shards was greater than 40. This means that the overall transaction throughput fails to reach the maximum value of the system, possibly due to the effective transaction arrival rate being limited to 3000 tps.

Then, we fix the number of shards to 16 and vary the transaction arrival rate to evaluate its impact on the system throughput. As shown in Figure 6b, when the transaction arrival rate increases from 500 tps to 1000 tps, the effective transaction throughput also increases by almost 500 tps. When the transaction arrival rate exceeds 1000 tps, the three types

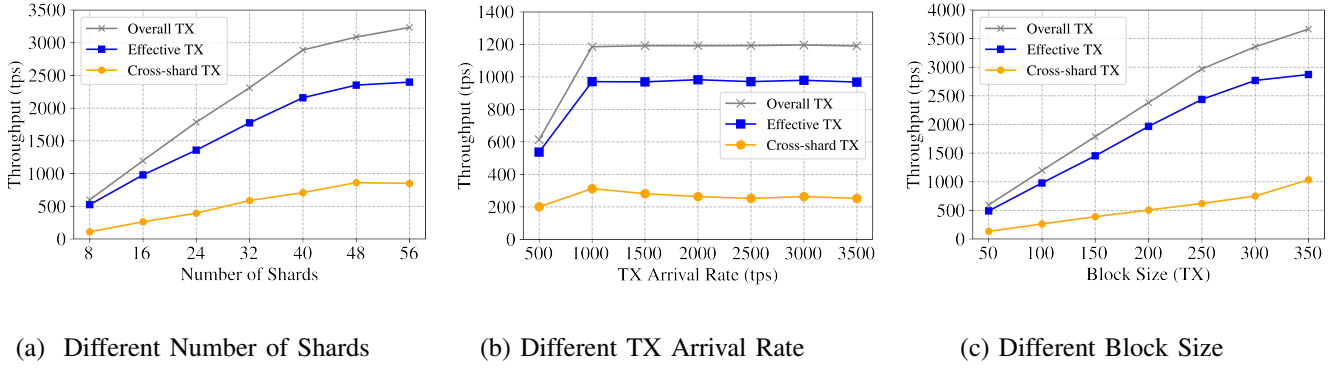


Fig. 6: System throughput with different (a) number of shards, (b) transaction arrival rate, and (c) block size

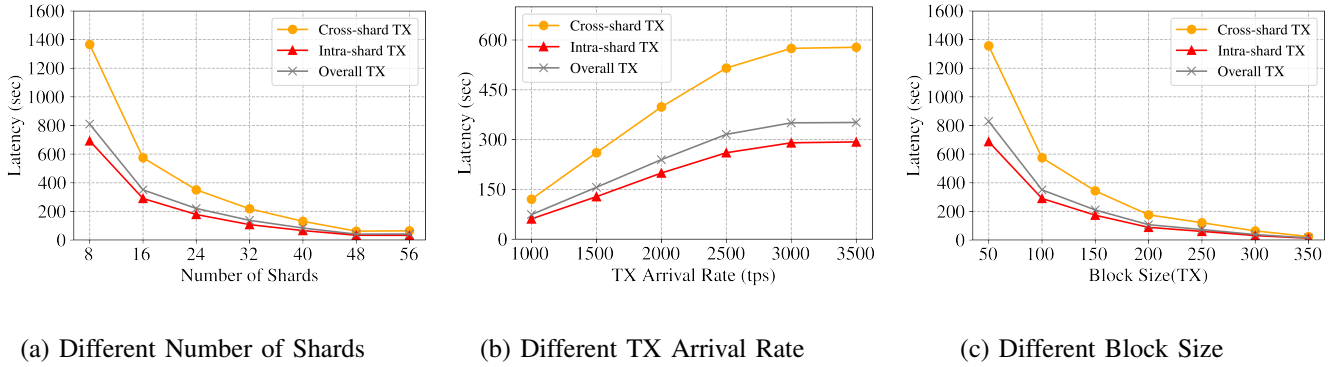


Fig. 7: Transaction processing latency with different (a) number of shards, (b) transaction arrival rates, and (c) block size

of throughput hardly change with the increase in transaction rate. This indicates that the transaction arrival rate has reached the maximum transaction processing rate of the system, and further increases in the transaction arrival rate do not improve the throughput of the system.

Finally, we evaluate the impact of the maximum block size on the system throughput. The system consists of 16 shards, each with a committee size of 10, and the transaction arrival rate is 3000 tps. As shown in Figure 6c, increasing the block size from 50 transactions to 350 transactions leads to higher throughput as larger blocks can pack more transactions. However, further increasing the maximum block size beyond 300 transactions leads to diminishing throughput gains. This is because, for sufficiently large blocks, throughput becomes limited by the transaction arrival rate rather than the block size.

C. Transaction Processing Latency

In this subsection, we evaluate the impact of the number of shards, transaction arrival rate, and block size on transaction processing latency.

We use three metrics to evaluate the transaction processing latency: intra-shard transaction processing latency, cross-shard transaction processing latency, and average transaction processing latency. The intra-shard transaction processing latency measures the delay between when an intra-shard transaction is added to the validated transaction pool and when the transaction is added to the blockchain by all honest validators within

the committee. The cross-shard transaction processing latency measures the delay between when a cross-shard transaction is added to the pending transaction pool and when it is finally added to the blockchain by honest validators of the input and output committees. The average transaction processing latency is the average delay for all transactions from being received by the shard to which they belong to being added to the blockchain.

First, we fix the Ethereum transaction arrival rate at 3000 tps, the committee size at 10, and the maximum block size at 100 transactions. As shown in Figure 7a, as the number of shards increases, all transaction processing latency decreases. This is because more shards process transactions in parallel, resulting in higher throughput and faster transaction processing rates. This is consistent with the theoretical analysis in Section IV. We can observe that the processing latency of cross-shard transactions is about twice the latency of transactions within a single shard. This is because processing a cross-shard transaction requires a round of intra-shard consensus within the input committee for the sub-transaction, followed by a round of intra-shard consensus within the output committee for the cross-shard transaction validation.

Second, we evaluate the impact of the transaction arrival rate on transaction processing latency. We fix the number of shards to 16 and the maximum block size to 100 transactions. As shown in Figure 7b, when the transaction arrival rate is between 1000 tps and 2500 tps, the transaction processing latency increases almost linearly with the transaction arrival

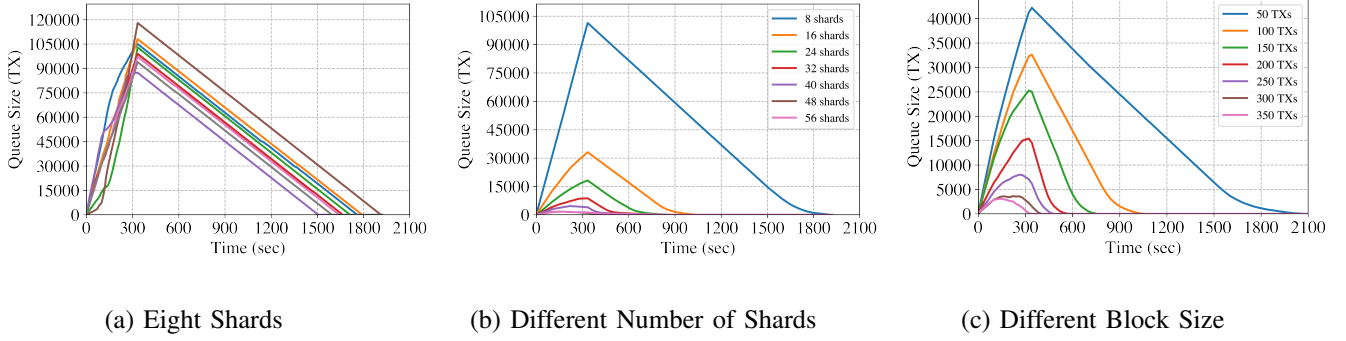


Fig. 8: Queue sizes with (a) eight shards, (b) different numbers of shards, and (c) different block size

rate. When the transaction arrival rate exceeds 3000 tps, the transaction processing latency becomes almost constant. A higher transaction arrival rate can quickly fill up the validated transaction pools, leading to fewer non-full blocks generated by validators. However, all blocks are full when the transaction rate reaches a certain threshold. At this point, increasing the transaction arrival rate does not improve the throughput, and the transaction processing latency remains almost constant.

Finally, we evaluate the impact of the block size on transaction processing latency. We set the number of shards in the system to 16, the committee size of each shard to 10, and the transaction arrival rate to 3000 tps. Figure 7c shows that the transaction processing latency decreases as the block size increases. This is because larger blocks allow more transactions to be added to the blockchain in each round of consensus, reducing the time that transactions must wait to be processed.

D. Queue Size

The validated transaction pools are modeled as queues (refer to Section IV). Figure 8a shows the number of transactions in the validated transaction pools in different shards when there are eight shards in the system, the number of validators in each shard is 10, and the transaction arrival rate is 3000 tps. We can observe that during the first 300 seconds, the number of transactions in all queues grows linearly over time at a similar rate, with little difference between the maximum values reached. This indicates that the transaction processing rate is less than the transaction arrival rate for the first 300 seconds, causing the queue size to increase rapidly. The maximum values of different shards are similar, indicating that our transaction allocation method can achieve system load balancing.

Next, we compare the sizes of validated transaction pools with different numbers of shards when the system processes one million transactions each time. Figure 8b shows that as the number of shards increases, the maximum queue size decreases, and the rate at which the queue grows decreases. This is because the number of transactions to be processed is the same, but an increase in the number of shards results in a decrease in the number of transactions assigned to each shard. After 300 seconds, the more the number of shards, the faster the queue size drops to zero. This is because more shards have

a higher transaction processing rate, allowing the transactions in the validated transaction pools to be quickly consumed.

Furthermore, Figure 8c shows the impact of the maximum block size (ranging from 50 transactions to 350 transactions) on the queue size. We can see that the larger the block size, the smaller the maximum queue size and the shorter the time to reach the maximum queue size. This is because larger blocks can pack more transactions per round, resulting in a slower increase in the number of transactions pending in the queue. Larger blocks also allow more transactions to be processed in a round of consensus, reducing the number of pending transactions and decreasing the maximum number of transactions in the queue.

E. Comparison of Sharding-based Blockchains

We compare X-shard with other sharding-based blockchains based on their cross-shard communication overhead of c cross-shard transactions, support for MIMO transactions, and transaction atomicity. We present the comparison results in Table I.

TABLE I: Comparison of sharding-based blockchains

Protocol	Cross-shard overhead	MIMO transaction	Atomicity
OmniLedger [9]	$O(cn)$	Y	Y
RapidChain [11]	$O(n)$	Y	N
AHL [12]	$O(n^2)$	Y	Y
Pyramid [13]	$O(n)$	Y	N
BrokerChain [28]	$O(n)$	N	Y
Monoxide [22]	$O(n)$	Y	Y
X-shard	$O(n)$	Y	Y

As shown in Table I, X-shard achieves the optimal cross-shard communication overhead of $O(n)$, which is significantly more efficient than protocols like OmniLedger and AHL. This reduces network congestion and improves system scalability. Moreover, X-shard supports efficient processing of MIMO transactions, enabling complex transaction workflows across shards. Additionally, X-shard guarantees transaction atomicity to prevent inconsistencies. With these advantages, X-shard represents a promising protocol for scalable, atomic cross-shard transaction processing in sharding-based blockchains.

VII. RELATED WORK

Elastico [5] is arguably the first sharding protocol for permissionless blockchains. Multiple committees pre-verify transactions and send results to a special committee called the final committee. The final committee packs the transactions into blocks and adds blocks to the blockchain. Elastico has only one blockchain, so there are no cross-shard transactions, and the throughput improvement from sharding is limited. OmniLedger [9] proposes using independent blockchains for each shard to increase the total system processing capacity proportionally with the number of shards. Transactions are randomly assigned to shards based on their IDs. OmniLedger relies on users to collect proofs from input committees and send them to output committees. The input is locked after the input committees generate the proofs. If the cross-shard transaction fails, users also need to send a message to the input committees to unlock the locked input. This user-driven solution places a burden on lightweight users [11] and requires all input committees to generate proofs for each user request individually, which is computation and communication expensive. RapidChain [11] requires the leader of the output committee to create several new transactions for each cross-shard transaction. Cross-shard transactions can be processed successfully if the leader is honest. However, RapidChain does not provide a solution for when a cross-shard transaction is successfully processed in some shards but fails in others. Dang *et al.* [12] propose AHL, using a 2PC protocol among all involved committees to reach consensus on the processing or non-processing of cross-shard transactions to ensure the atomicity of cross-shard transactions. The 2PC protocol relies on a reference committee, which can be a performance bottleneck for processing cross-shard transactions [29], [30].

Nguyen *et al.* [31] points out that allocating transactions to shards based on transaction IDs results in most transactions being cross-shard transactions. They propose OptChain, an optimal transaction allocation algorithm only applicable to the Unspent Transaction Output (UTXO) model [32]. OptChain uses historical transactions as nodes to construct the transaction network graph and introduces the concept of ‘temporal fitness’ as a score to evaluate the appropriateness of placing transactions in each shard. Besides, OptChain does not provide a cross-shard transaction processing protocol [32]. BrokerChain [28] is designed for account-based blockchains. It relies on brokers with accounts in multiple shards to process cross-shard transactions. A broker can only process cross-shard transactions one by one sequentially, and it cannot process cross-shard transactions with multiple inputs and multiple outputs. Pyramid [13] is a layered sharding-based blockchain. Some nodes can store the state of multiple shards and process cross-shard transactions involving those shards. It is prone to system deadlocks because nodes may always wait for resources to be released during the block commit phase.

PolyShard [33] applies coded computing to sharding for linear scalability and security. It splits data into coded shards, enabling parallel processing and resilience to compromised shards. While both PolyShard and X-shard aim to scale blockchains, PolyShard uses coding-theoretic approaches for

efficiency and security guarantees. In contrast, X-shard focuses on an optimistic strategy and transaction allocation to optimize cross-shard processing. Further research can explore combining the coded computing approach of PolyShard with X-shard’s optimistic transaction processing strategy for greater scalability.

VIII. CONCLUSION

We present X-shard, a blockchain system that takes an optimistic strategy to process multi-input multi-output cross-shard transactions. The validation of cross-shard transactions is converted into intra-shard validation of multiple sub-transactions. Valid sub-transactions are aggregated and signed in batches to achieve high cross-shard transaction processing efficiency. Invalid cross-shard transactions are rolled back to ensure transaction atomicity. Our performance analysis shows that with the increase in the number of shards, the effective transaction throughput of the blockchain system increases almost linearly, and the transaction processing latency decreases. Comprehensive experiments validate the performance analysis, demonstrating that X-shard achieves high effective throughput and low transaction processing latency.

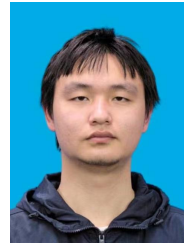
REFERENCES

- [1] Y. Lu, Z. Lu, and Q. Tang, “Bolt-dumbo transformer: Asynchronous consensus as fast as the pipelined bft,” in *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*, 2022, pp. 2159–2173.
- [2] X. Luo, Z. Xu, K. Xue, Q. Jiang, R. Li, and D. Wei, “Scalacert: Scalability-oriented pki with redactable consortium blockchain enabled” on-cert” certificate revocation,” in *2022 IEEE 42nd International Conference on Distributed Computing Systems (ICDCS)*. IEEE, 2022, pp. 1236–1246.
- [3] Z. Xu, X. Luo, K. Xue, D. Wei, and R. Li, “Seredact: Secure and efficient redactable blockchain with verifiable modification,” in *Proceedings of the 2023 IEEE 42nd International Conference on Distributed Computing Systems (ICDCS)*. IEEE, 2023, pp. 1–12.
- [4] S. Nakamoto, “Bitcoin: A peer-to-peer electronic cash system,” 2008, <http://www.bitcoin.org/bitcoin.pdf>.
- [5] L. Luu, V. Narayanan, C. Zheng, K. Baweja, and P. Saxena, “A secure sharding protocol for open blockchains,” in *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*. ACM, 2016, pp. 17–30.
- [6] J. Xu, C. Wang, and X. Jia, “A survey of blockchain consensus protocols,” *ACM Computing Surveys*, 2023. [Online]. Available: <https://doi.org/10.1145/3579845>
- [7] J. Xu, Q. Xie, S. Peng, C. Wang, and X. Jia, “Adaptchain: Adaptive scaling blockchain with transaction deduplication,” *IEEE Transactions on Parallel and Distributed Systems*, 2023.
- [8] C. Xu, C. Zhang, J. Xu, and J. Pei, “Slimchain: Scaling blockchain transactions through off-chain storage and parallel processing,” *Proceedings of the VLDB Endow.*, vol. 14, no. 11, pp. 2314–2326, 2021.
- [9] E. Kokoris-Kogias, P. Jovanovic, L. Gasser, N. Gailly, and E. Syta, “OmniLedger: A Secure, Scale-Out, Decentralized Ledger via Sharding,” in *Proceedings of the 2018 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2018, pp. 583–598.
- [10] C. Chen, Q. Ma, X. Chen, and J. Huang, “User distributions in shard-based blockchain network: Queueing modeling, game analysis, and protocol design,” in *Proceedings of the Twenty-second International Symposium on Theory, Algorithmic Foundations, and Protocol Design for Mobile Networks and Mobile Computing*, pp. 221–230.
- [11] M. Zamani, M. Movahedi, and M. Raykova, “Rapidchain: Scaling blockchain via full sharding,” in *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*, 2018, pp. 931–948.
- [12] H. Dang, T. T. A. Dinh, D. Loghin, E.-C. Chang, Q. Lin, and B. C. Ooi, “Towards scaling blockchain systems via sharding,” in *Proceedings of the 2019 international conference on management of data*, 2019, pp. 123–140.

- [13] Z. Hong, S. Guo, P. Li, and W. Chen, "Pyramid: A layered sharding blockchain system," in *Proceedings of the 2021 IEEE Conference on Computer Communications (INFOCOM)*. IEEE, 2021, pp. 1–10.
- [14] J. Doerner, Y. Kondi, E. Lee, and A. Shelat, "Threshold ecDSA from ecDSA assumptions: the multiparty case," in *Proceedings of the 2019 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2019, pp. 1051–1066.
- [15] V. Shoup, "Practical threshold signatures," in *International Conference on the Theory and Applications of Cryptographic Techniques*. Springer, 2000, pp. 207–220.
- [16] B. Guo, Z. Lu, Q. Tang, J. Xu, and Z. Zhang, "Dumbo: Faster asynchronous bft protocols," in *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security*, 2020, pp. 803–818.
- [17] M. Yin, D. Malkhi, M. K. Reiter, G. G. Gueta, and I. Abraham, "Hot-stuff: Bft consensus with linearity and responsiveness," in *Proceedings of the 2019 ACM Symposium on Principles of Distributed Computing*, 2019, pp. 347–356.
- [18] G. Karypis and V. Kumar, "A fast and high quality multilevel scheme for partitioning irregular graphs," *SIAM Journal on Scientific Computing*, vol. 20, no. 1, pp. 359–392, 1998.
- [19] F. Bourse, M. Lelarge, and M. Vojnovic, "Balanced graph edge partition," in *Proceedings of the 20th ACM SIGKDD international conference on Knowledge discovery and data mining*, 2014, pp. 1456–1465.
- [20] A. Awadelkarim and J. Ugander, "Prioritized restreaming algorithms for balanced graph partitioning," in *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, 2020, pp. 1877–1887.
- [21] P. Maymounkov and D. Mazières, "Kademlia: A peer-to-peer information system based on the xor metric," in *Proceedings of the 2002 International Workshop on Peer-to-Peer Systems*. Springer, 2002, pp. 53–65.
- [22] J. Wang and H. Wang, "Monoxide: Scale out blockchains with asynchronous consensus zones," in *Proceedings of the 16th USENIX symposium on networked systems design and implementation (NSDI 19)*, 2019, pp. 95–112.
- [23] D. Margo and M. Seltzer, "A scalable distributed graph partitioner," *Proceedings of the VLDB Endowment*, vol. 8, no. 12, pp. 1478–1489, 2015.
- [24] M. Castro and B. Liskov, "Practical byzantine fault tolerance," in *Proceedings of the Third Symposium on Operating Systems Design and Implementation*. The Advanced Computing Systems Association, 1999, pp. 173–186.
- [25] U. N. Bhat, *An introduction to queueing theory: modeling and analysis in applications*. Springer, 2008, vol. 36.
- [26] G. Wood, "Ethereum: A secure decentralised generalised transaction ledger," *Ethereum project yellow paper*, vol. 151, pp. 1–32, 2014.
- [27] P. Zheng, Z. Zheng, J. Wu, and H.-N. Dai, "Xblock-eth: Extracting and exploring blockchain data from ethereum," *IEEE Open Journal of the Computer Society*, vol. 1, pp. 95–106, 2020.
- [28] H. Huang, X. Peng, J. Zhan, S. Zhang, Y. Lin, Z. Zheng, and S. Guo, "Brokerchain: A cross-shard blockchain protocol for account/balance-based state sharding," in *Proceedings of the 2022 IEEE Conference on Computer Communications (INFOCOM)*. IEEE, 2022, pp. 1968–1977.
- [29] M. J. Amiri, D. Agrawal, and A. El Abbadi, "Sharper: Sharding permissioned blockchains over network clusters," in *Proceedings of the 2021 International Conference on Management of Data*, 2021, pp. 76–88.
- [30] J. Hellings and M. Sadoghi, "Byshard: Sharding in a byzantine environment," *Proceedings of the VLDB Endowment*, vol. 14, no. 11, pp. 2230–2243, 2021.
- [31] L. N. Nguyen, T. D. T. Nguyen, T. N. Dinh, and M. T. Thai, "Optchain: Optimal transactions placement for scalable blockchain sharding," in *Proceedings of the 39th International Conference on Distributed Computing Systems (ICDCS)*, 2019, pp. 525–535.
- [32] S. Das, V. Krishnan, and L. Ren, "Efficient cross-shard transaction execution in sharded blockchains," *CoRR*, vol. abs/2007.14521, 2020, <https://arxiv.org/abs/2007.14521>.
- [33] S. Li, M. Yu, C.-S. Yang, A. S. Avestimehr, S. Kannan, and P. Viswanath, "Polyshard: Coded sharding achieves linearly scaling efficiency and security simultaneously," *IEEE Transactions on Information Forensics and Security*, vol. 16, pp. 249–261, 2020.



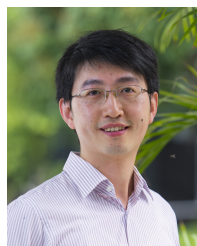
Jie Xu received the B.S. degree from the Department of Information Security, University of Science and Technology of China (USTC) in July 2017, and the M.S. degree from the Department of Electronic Engineering and Information Science, USTC in 2020. She is currently pursuing the Ph.D. degree with the Department of Computer Science, City University of Hong Kong. Her research interests include network security and cryptography.



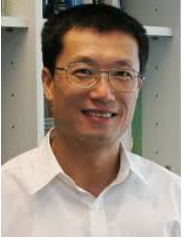
Yulong Ming received the B.S. degree from the School of Computer Science and Technology, University of Science and Technology of China (USTC) in July 2022. He is currently pursuing the Ph.D. degree with the Department of Computer Science, City University of Hong Kong. His research interest is network security including metadata privacy.



Zihan Wu received a Bachelor's degree in Physics and a dual major in Mathematics from the School for the Gifted Young, University of Science and Technology of China (USTC) in June 2020. He is currently pursuing a Ph.D. degree in the Department of Electrical Engineering at the City University of Hong Kong. His research interests include pattern recognition and computer vision.



Cong Wang (Fellow, IEEE) is currently a Professor with the Department of Computer Science, City University of Hong Kong. He is also with the City University of Hong Kong Shenzhen Research Institute. His research interests include data and network security, blockchain and decentralized applications, and privacy-enhancing technologies. He is a member of ACM. He has been one of the Founding Member of the Young Academy of Sciences of Hong Kong since 2017. He was conferred the RGC Research Fellow in 2021. He received the Outstanding Researcher Award (junior faculty) in 2019, the Outstanding Supervisor Award in 2017, and the President's Awards in 2016 and 2019, all from the City University of Hong Kong. He was a co-recipient of the Best Paper Award of IEEE ICDCS 2020, ICPADS 2018, MSN 2015, the Best Student Paper Award of IEEE ICDCS 2017, and the IEEE INFOCOM Test of Time Paper Award in 2020. His research has been supported by multiple government research fund agencies, including the National Natural Science Foundation of China, the Hong Kong Research Grants Council, and the Hong Kong Innovation and Technology Commission. He served as the TPC co-chair for a number of IEEE conferences and workshops. He has served as an Associate Editor for IEEE Transactions on Dependable and Secure Computing, IEEE Transactions on Services Computing, IEEE Internet of Things Journal, IEEE Networking Letters, and the Journal of Blockchain Research.



Xiaohua Jia (Fellow, IEEE) received the BSc and MEng degrees in 1984 and 1987, respectively, from the University of Science and Technology of China, and the DSc degree in 1991 in information science from the University of Tokyo. He is currently the chair professor with the Department of Computer Science at the City University of Hong Kong. His research interests include cloud computing and distributed systems, computer networks, wireless sensor networks and mobile wireless networks. He is an editor of the *IEEE Transactions on Parallel and Distributed Systems* (2006-2009), *Wireless Networks*, *Journal of World Wide Web*, *Journal of Combinatorial Optimization*, etc. He is the general chair of ACM MobiHoc 2008, TPC co-chair of IEEE MASS 2009, area-chair of IEEE INFOCOM 2010, TPC co-chair of IEEE GlobeCom 2010-Ad Hoc and Sensor Networking Symposium, and Panel co-chair of IEEE INFOCOM 2011. He is fellow of the IEEE Computer Society.

APPENDIX

A. Proof of Theorem 1

Theorem 1. *The maximum effective transaction throughput of the system grows linearly with the number of shards.*

Proof. We assume that, on average, each transaction in the system involves n accounts. For example, in the Ethereum system, transactions have only one input and one output, so $n = 2$. In a blockchain system where transactions are multiple-input and multiple-output, n can be 2, 3, or more. The probability that a transaction with n accounts involves m shards is represented by α_m .

All involved shards need to process an effective transaction once. If a transaction involves m shards, the number of times the transaction needs to be processed is m . Thus, the expected number of times an effective transaction needs to be processed by different shards is $\sum_{m=1}^n m\alpha_m$. If the maximum effective transaction throughput of the system, or the maximum rate at which the system can process effective transactions is τ , then the maximum overall transaction processing rate of the system is $\tau \sum_{m=1}^n m\alpha_m$.

When the system reaches an equilibrium of transaction processing supply and demand, the maximum overall transaction processing rate of the system is equal to the sum of the maximum overall transaction processing rates of each shard μ , which is $k\mu$. Thus, we can get the following:

$$k\mu = \tau \sum_{m=1}^n m\alpha_m. \quad (9)$$

Let χ_i be the random variable where

$$\chi_i = \begin{cases} 1, & \text{if the transaction involves shard } i \\ 0, & \text{if the transaction does not involve shard } i \end{cases} \quad (10)$$

The probability that any account is assigned to shard i is $\frac{1}{k}$. Correspondingly, the probability that an account is not assigned to i is $1 - \frac{1}{k}$.

Thus, the probability that a transaction does not involve shard i is:

$$P(\chi_i = 0) = \left(1 - \frac{1}{k}\right)^n = \left(\frac{k-1}{k}\right)^n. \quad (11)$$

Correspondingly, the probability that a transaction involves shard i is:

$$P(\chi_i = 1) = 1 - P(\chi_i = 0) = 1 - \left(\frac{k-1}{k}\right)^n. \quad (12)$$

The expectation is

$$\begin{aligned} E(\chi_i) &= 0 \cdot P(\chi_i = 0) + 1 \cdot P(\chi_i = 1) \\ &= 1 - \left(\frac{k-1}{k}\right)^n. \end{aligned} \quad (13)$$

Let ξ denote the number of shards processing a transaction with n accounts.

$$\xi = \chi_1 + \chi_2 + \cdots + \chi_k. \quad (14)$$

According to the linearity of expectation,

$$\begin{aligned} E(\xi) &= E(\chi_1) + E(\chi_2) + \dots + E(\chi_k) \\ &= k \cdot \left[1 - \left(\frac{k-1}{k} \right)^n \right]. \end{aligned} \quad (15)$$

According to the definition of expectation,

$$E(\xi) = \sum_{m=1}^n \alpha_m m. \quad (16)$$

Combining Eq. (15) and Eq. (16), we can get

$$E(\xi) = k \cdot \left[1 - \left(\frac{k-1}{k} \right)^n \right] = \sum_{m=1}^n \alpha_m m. \quad (17)$$

Combining Eq. (17) with Eq. (9), we can get

$$\begin{aligned} \tau &= \frac{k\mu}{\sum_{m=1}^n \alpha_m m} = \frac{k\mu}{k \cdot \left[1 - \left(\frac{k-1}{k} \right)^n \right]} \\ &= \frac{\mu}{1 - \left(\frac{k-1}{k} \right)^n} \\ &= \frac{\mu k^n}{k^n - (k-1)^n}. \end{aligned} \quad (18)$$

Because

$$\lim_{k \rightarrow \infty} \frac{k^n - (k-1)^n}{nk^{n-1}} = \lim_{k \rightarrow \infty} \frac{k^n - k^n + nk^{n-1} + o(k^{n-1})}{nk^{n-1}} = 1, \quad (19)$$

combining Eq. (18) and Eq. (19), we can get:

$$\lim_{k \rightarrow \infty} \frac{\mu k}{n\tau} = \lim_{k \rightarrow \infty} \frac{k^n - (k-1)^n}{nk^{n-1}} = 1. \quad (20)$$

Therefore, when $k \rightarrow \infty$, $\tau \sim \frac{\mu}{n} k$.

When the number of shards k is large enough, the blockchain system that randomly assigns transactions to shards can also make the effective throughput of the system τ increase linearly with the number of shards. $\square$

B. Proof of Theorem 2

Theorem 2. *The average transaction processing latency of the system decreases as the number of shards increases.*

Proof. Effective transactions consist of intra-shard transactions and cross-shard transactions. For a system where accounts are randomly assigned to shards, the probability that a transaction involving n accounts is an intra-shard transaction P_{in} is

$$P_{\text{in}} = \frac{1}{k^{n-1}}. \quad (21)$$

Correspondingly, the probability that a transaction involving n accounts is a cross-shard transaction P_{cr} is

$$P_{\text{cr}} = 1 - \frac{1}{k^{n-1}}. \quad (22)$$

Therefore, the arrival rate of intra-shard transactions λ_{in} and cross-shard transactions λ_{cr} in the system are

$$\begin{aligned} \lambda_{\text{in}} &= \lambda P_{\text{in}} = \frac{\lambda}{k^{n-1}}, \\ \lambda_{\text{cr}} &= \lambda P_{\text{cr}} = \lambda \left(1 - \frac{1}{k^{n-1}} \right), \end{aligned} \quad (23)$$

where λ is the arrival rate of effective transactions in the system.

We assume that, on average, each transaction in the system involves n accounts, the number of input shards is s_1 , and the number of output shards is s_2 . Then, the arrival rate of sub-transactions processed by all input shards is

$$\lambda_{\text{sub}} = s_1 \lambda_{\text{cr}} = s_1 \lambda \left(1 - \frac{1}{k^{n-1}} \right). \quad (24)$$

When the system attains its statistical equilibrium, the departure process of a $M/M/1$ queue is the same Poisson as the arrival process [25]. Thus, the arrival rate of cross-shard transactions processed by all output shards is

$$\lambda_{\text{crou}} = s_2 \lambda_{\text{sub}} = s_1 s_2 \lambda \left(1 - \frac{1}{k^{n-1}} \right). \quad (25)$$

Due to the homogeneity assumption of shards, all shards have the same transaction arrival rate. Each shard is an $M/M/1$ queue whose transaction arrival rate is $\frac{1}{k}$ of the transaction arrival rate of the total system. An arbitrary shard $i \in \{1, 2, \dots, k\}$ processes three types of transactions: intra-shard transactions, cross-shard transactions with input accounts within the shard, and cross-shard transactions with output accounts within the shard. Thus, the transaction arrival rate $\lambda_0(k)$ of a shard is:

$$\begin{aligned} \lambda_0(k) &= \frac{\lambda_{\text{in}} + \lambda_{\text{sub}} + \lambda_{\text{crou}}}{k} \\ &= \lambda \frac{1}{k^n} + \lambda s_1 (1 + s_2) \left(\frac{1}{k} - \frac{1}{k^n} \right). \end{aligned} \quad (26)$$

The growth of λ_0 as k grows can be expressed as

$$\begin{aligned} &\lambda_0(k+1) - \lambda_0(k) \\ &= \lambda \frac{-b(k+1)^{n-1}k^{n-1} + (b-1)[(k+1)^n - k^n]}{(k+1)^n k^n}, \end{aligned} \quad (27)$$

where $b = s_1(1 + s_2) > 0$.

As $k \rightarrow \infty$,

$$\lambda_0(k+1) - \lambda_0(k) < 0. \quad (28)$$

Thus, when k is sufficiently large, λ_0 will decrease as k increases.

According to the property of $M/M/1$ [25], the expectation of the transaction processing latency W_p is

$$W_p = \frac{1}{\mu - \lambda_0}. \quad (29)$$

Because

$$\frac{dW_p}{d\lambda_0} = \frac{1}{(\mu - \lambda_0)^2} > 0, \quad (30)$$

W_p increases as the transaction arrival rate λ_0 increases, and W_p decreases as λ_0 decreases.

Combining this result with the conclusion from Ineq. (28), we can conclude that when the number of shards k is sufficiently large, the transaction arrival rate λ_0 decreases as k increases, and the transaction processing latency W_p also decreases. $\square$

C. Proof of Theorem 3

Theorem 3. *For each epoch with an unbiased and random committee formation algorithm, as long as $\beta < \frac{1}{3}$ and the number of validators in the committee n is sufficiently large, any committee is honest with an overwhelming probability.*

Proof. The committee formation algorithm can be modeled as random sampling. Let X denote a random variable representing the number of malicious validators selected in the committee. The distribution of X follows the hypergeometric distribution with parameters n, N, M where n is the number of validators in the committee, N is the total number of validators, and $M = \beta N$ is the total number of malicious validators. The probability that a committee of size n contains k malicious validators is:

$$P(X = k) = \frac{\binom{M}{k} \binom{N-M}{n-k}}{\binom{N}{n}}, \quad (31)$$

where $\binom{M}{k}$ is the binomial coefficient. From the hypergeometric distribution, the expectation and variance of X are given by

$$E(X) = n\beta, \quad (32)$$

$$Var(X) = \frac{nM(N-n)(N-M)}{N^2(N-1)} = n\beta(1-\beta) \frac{N-n}{N-1}. \quad (33)$$

The probability p_C^A that the number of malicious validators exceeds $\frac{1}{3}$ of the total number of validators in the committee is:

$$p_C^A = P(X \geq \frac{1}{3}n) = P(X - \beta n \geq n(\frac{1}{3} - \beta)). \quad (34)$$

According to Chebyshev's inequality,

$$\begin{aligned} P(X - \beta n \geq n(\frac{1}{3} - \beta)) &\leq \frac{n\beta(1-\beta) \frac{N-n}{N-1}}{n^2(\frac{1}{3} - \beta)^2} \\ &= \frac{\beta(1-\beta)}{(\frac{1}{3} - \beta)^2(N-1)} (\frac{N}{n} - 1). \end{aligned} \quad (35)$$

Given $\eta > 0$, $n_0 = \left\lceil \frac{N}{\frac{(\frac{1}{3}-\beta)^2\eta}{\beta(1-\beta)} + 1} \right\rceil$, for any $n \geq n_0$, Ineq. (35) satisfies:

$$P(X - \beta n \geq n(\frac{1}{3} - \beta)) \leq \frac{\beta(1-\beta)}{(\frac{1}{3} - \beta)^2(N-1)} (\frac{N}{n} - 1) \quad (37)$$

$$\leq \frac{\beta(1-\beta)}{(\frac{1}{3} - \beta)^2(N-1)} (\frac{N}{n_0} - 1) \quad (38)$$

$$\leq \eta. \quad (39)$$

Combining Ineq.(34), Ineq.(35) and Ineq. (37), we have $p_C^A \leq \eta$, which means that the probability that the number of malicious validators exceeds $\frac{1}{3}$ of the total number of validators in the committee is less than η . Therefore, the committee is honest as long as $\beta < \frac{1}{3}$ and the number of validators in the committee n is sufficiently large. $\square$